1 Лекция 10. Дерандомизация color coding и random separation

1.1 Что означает убрать случайность

В color coding случайность помогает найти удачную раскраску вершин пути. Динамика при уже заданных цветах работает точно. **Дерандомизация (derandomization)** заменяет случайные попытки детерминированным списком раскрасок с гарантией: для каждого возможного решения хотя бы одна раскраска списка подходит. После проверки всего списка отрицательный ответ становится достоверным.

Не нужно заранее знать путь, для которого нужна удачная раскраска. Именно поэтому список должен одновременно покрывать все возможные k-элементные множества вершин, но разными функциями. Полный перебор k^n раскрасок даёт гарантию, однако не FPT-время. Задача главы — объяснить, каким должно быть малое семейство, как из него получается алгоритм и почему доказательство существования ещё не равно построению.

Вторая тема — **случайное разделение (random separation)**. Вместо разных цветов внутри решения назначаем один цвет самому решению, а другой его границе. В удачной попытке решение выделяется целой компонентой, которую легко найти. Для этого нужно ограничить размер границы; мы будем использовать максимальную степень графа d и параметр (k, d).

Неравенство объединения (union bound) утверждает $\Pr(\bigcup_i A_i) \le \sum_i \Pr(A_i)$ для любых событий. Доказательство: индикатор объединения не больше суммы индикаторов; берём ожидания. Независимость для этого не требуется. Напротив, формула $(1-p)^r$ для r неудач ниже использует независимость повторений.

1.2 Perfect hash family

(n, k)-совершенное семейство хеш-функций (perfect hash family) — набор $\mathcal{H}$ функций $h : [n] \to [k]$, такой, что для каждого k-элементного $S \subseteq [n]$ хотя бы одна $h \in \mathcal{H}$ инъективна на S. Порядок кванторов: для каждого S существует своя функция; одной функции, инъективной сразу на всех k-наборах при $n > k$, обычно нет.

Построив такое семейство, запускаем colorful-DP для всех его функций. Если k-путь существует, один набор его вершин раскрасится инъективно, и алгоритм найдёт путь. Если все попытки неудачны, ответ NO уже достоверен. Время — время построения семейства плюс $|\mathcal{H}|2^k N^{O(1)}$.

1.3 Малое существует — не значит, что умеем быстро строить

Выберем r независимых случайных функций. Для фиксированного S вероятность, что ни одна не инъективна, не больше $e^{-re^{-k}}$. Всего $\binom{n}{k} \le n^k$ наборов. По union bound вероятность существования непокрытого набора не больше $n^k e^{-re^{-k}}$. При $r > e^k(k \ln n + 1)$ она меньше 1, значит, хорошее семейство существует.

Но проверка всех $\binom{n}{k}$ наборов была бы XP, а перебор всех семейств ещё дороже. Это существовательное доказательство не даёт нужной эффективной дерандомизации.

Используемая внешняя теорема (Cygan et al., теорема 5.18, с. 118): при $n, k \ge 2$, $k \le n$, семейство размера $e^k k^{O(\log k)} \log n$ можно детерминированно построить за $e^k k^{O(\log k)} n \log n$. Конструкцию через splitters здесь не доказываем; $k = 1$ обрабатывается отдельно. Следствие, уже с полным выводом: k-Path имеет детерминированный алгоритм $(2e)^k k^{O(\log k)} N^{O(1)}$. Применение теоремы и её доказательство — разные учебные цели.

1.4 Самодостаточная конструкция, достаточная для FPT

Для самого факта детерминированной FPT-разрешимости можно обойтись без теоремы о быстрых splitters. Приведём более медленную, но полностью явную конструкцию. Она также показывает, почему степень логарифма, зависящая от параметра, иногда допустима.

Пронумеруем $n \geq 2$ объектов различными двоичными строками длины $\ell = \lceil \log_2 n \rceil$. Пусть $k \geq 2$, $a = \binom{k}{2}$. Для любого фиксированного множества S из k объектов у каждой пары есть различающий их бит. Выбрав по одному такому биту для каждой пары, получим множество позиций J размера не больше a, проекция на которое инъективна на S.

Переберём все $J \subseteq [\ell]$ размера до a. Для каждого переберём все функции $g : \{0,1\}^{|J|} \to [k]$ и включим в семейство раскраску $h(x) = g(x|_J)$. Для подходящего J все проекции элементов S различны; среди перебираемых g есть функция, назначающая им разные цвета. Следовательно, получено perfect hash family, причём алгоритм построения указан полностью.

Число вариантов не больше $(a+1)(\ell+1)^a k^{2^a}$. Докажем, что это $F(k)n$, а не только XP. При $x = \ln n \geq 0$ максимум $x^a e^{-x}$ равен $(a/e)^a$ (дифференцируем, максимум при $x = a$). Следовательно, $(\ln n)^a \leq (a/e)^a n$. Замена $\ln n$ на $\ell + 1 \leq C(\ln n + 1)$ меняет только множитель, зависящий от a. Поэтому $(\ell+1)^a \leq F_1(k)n$ для вычислимой F_1. Явная запись всех раскрасок занимает $F_2(k)n^2$ времени. Запуск colorful-DP для каждой сохраняет FPT-оценку, хотя зависимость от k огромна. При $k = 1$ достаточно одной постоянной функции, при $k > n$ исходный k-Path сразу NO.

Таким образом, конструктивная дерандомизация доказана здесь целиком. Теорема 5.18 нужна для существенно лучшей зависимости от параметра, а не для логической полноты базового FPT-алгоритма.

1.5 Random separation: изолируем целое решение

Рассмотрим Subgraph Isomorphism для связного простого образца H на k вершинах в графе G максимальной степени d. Требуется обычный подграф, не обязательно индуцированный. Параметр — (k, d). При $k = 1$ ответ зависит лишь от непустоты G; далее $k \geq 2$.

Независимо красим каждое ребро G в красный/синий с вероятностями $1/2$. Зафиксируем найденную в воображении копию $\widehat{H}$: красим все её рёбра красным, а все остальные рёбра, инцидентные её вершинам, синим. Последние включают и лишние хорды внутри выбранных вершин. Всего затронутых рёбер не больше dk, поэтому вероятность события не меньше 2^{-dk}.

При этом событии $\widehat{H}$ становится целой связной компонентой красного графа $G_R = (V, E_R)$, и эта компонента содержит ровно нужные рёбра. Алгоритм перечисляет красные компоненты на k вершинах и проверяет изоморфизм с H перебором $k!$ биекций. Найденное совпадение даёт настоящий подграф исходного G; ложного YES нет. За $O(2^{dk} \log(1/\delta))$ повторений получаем ошибку не больше δ и время $2^{dk} k! N^{O(1)} \log(1/\delta)$.

Непосредственный алгоритм описан для **связного** H. Для несвязного образца понадобилось бы согласованно выбрать разные красные компоненты, например через двудольное matching; одного теста компоненты уже недостаточно.

1.6 Универсальные множества

(m, t)-универсальное семейство — набор двоичных раскрасок m позиций, реализующий любой шаблон нулей и единиц на любом наборе из t позиций. Для random separation нужно реализовать предписанные цвета на не более dk рёбрах. Perfect hash family решает другой запрос — инъективность, а не конкретный двоичный шаблон.

Теорема 5.20 источника даёт эффективное универсальное семейство размера

$$2^t t^{O(\log t)} \log m.$$

Здесь она также внешний факт. Если $t > m$, можно перебрать все $2^m \leq 2^t$ раскрасок. Это превращает random separation в детерминированный FPT по (k, d).

Самопроверка: где требуется ограниченная степень? Почему нельзя красить только вершины и проверять индуцированную компоненту для произвольного неиндуцированного образца? Почему независимый запуск и перебор универсального семейства дают разные виды гарантии?

1.7 Явная дерандомизация разделения

Ту же конструкцию можно использовать для универсального семейства. Пронумеруем m позиций двоичными строками. Для $t \geq 2$ переберём наборы до $\binom{t}{2}$ битовых позиций J, а затем все булевы функции на проекциях: $g : \{0,1\}^{|J|} \to \{0,1\}$. На любом фиксированном наборе из t позиций найдётся инъективная проекция, а подходящая g реализует любой требуемый шаблон. Оценка времени снова $F(t)m^{O(1)}$ тем же поглощением степени логарифма. При $t = 1$ нужны две постоянные раскраски; при $t = 0$ достаточно одной; при $t > m$ перебираем все раскраски.

Это даёт полностью объяснённый детерминированный вариант random separation по (k, d) с $t = dk$. Более быстрый размер $2^t t^{O(\log t)} \log m$ остаётся дополнительной оценкой из сформулированной выше теоремы.

Для примера совершенного семейства на трёх объектах и двух цветах подходят функции $(1, 1, 2)$ и $(1, 2, 1)$. Каждая пара получает разные цвета хотя бы в одной из них. Первая функция не различает объекты 1 и 2, вторая не различает 1 и 3, но нам не требуется одна функция, которая различает сразу всё.

1.8 Практика

Условия задач 10.1–10.5. Решения открывайте после своей попытки.

1.9 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §5.3, с. 106–107; §5.6, с. 118–120. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.